



Facultad de
INFORMÁTICA



UNIVERSIDAD
NACIONAL
DE LA PLATA

Informe de Avance

A4 - Reconocimiento de gesto estático

Caciani Toniolo, Melina 02866/1

Chanquía, Joaquín 02887/7

Ollier, Gabriel 02958/4

UNLP

Facultad de Informática

Taller de Proyecto II

12 de octubre de 2025

ÍNDICE

INTRODUCCIÓN	3
OBJETIVO	4
Objetivo general.....	4
Funcionalidad y requerimientos.....	4
MATERIALES Y PRESUPUESTO.....	5
ESQUEMA GRÁFICOS DEL PROYECTO.....	5
GRADO DE AVANCE, PROBLEMAS Y SOLUCIONES	6
Módulo del auto	6
Hardware.....	6
Firmware	6
Módulo ESP32-CAM.....	11
Hardware.....	11
Interfaz web	11
Reconocimiento de gestos.....	12
DOCUMENTACIÓN RELACIONADA	14
Investigación y trabajo inicial	14
Módulo del auto	14
Hardware.....	14
Firmware	15
Módulo ESP32-CAM.....	16
Hardware.....	16
Interfaz web	16
Repositorio.....	17

INTRODUCCIÓN

En los últimos años, el reconocimiento de gestos mediante visión por computadora se ha convertido en una herramienta fundamental para el desarrollo de interfaces más naturales e intuitivas. Este tipo de sistemas permite una interacción fluida entre el ser humano y dispositivos electrónicos, liberando al usuario de la necesidad de controles físicos y aprovechando la capacidad de la comunicación no verbal a través de los movimientos de la mano.

El presente proyecto surge en este contexto y corresponde a los requisitos de la materia Taller de Proyecto II, cursada en el último semestre de la carrera de Ingeniería en Computación. Como parte de este desafío final, se busca aplicar y sintetizar los conocimientos adquiridos a lo largo de la formación académica.

El objetivo principal de este trabajo es implementar un sistema capaz de reconocer gestos estáticos de la mano utilizando TensorFlow Lite integrado en un ESP32 con un módulo de cámara y vincular este reconocimiento con el control a distancia de un vehículo robótico.

El desafío técnico de este proyecto consiste en la optimización del procesamiento en un hardware de recursos limitados, lo que exige soluciones de software eficientes para garantizar la detección precisa de la palma de la mano, la correcta interpretación de los 21 puntos de referencia y la clasificación confiable de los gestos. La culminación de este proyecto no solo representa la adquisición de habilidades técnicas, sino también la demostración de la capacidad de llevar un concepto complejo desde su concepción hasta su implementación funcional, sirviendo como una prueba final de nuestras capacidades como futuros ingenieros en computación.

OBJETIVO

Objetivo general

Se busca desarrollar con TensorFlow Lite y un ESP32 con CAM, un sistema que pueda reconocer los movimientos de los dedos de una mano humana y traducir dichos gestos en comandos de movimiento para un vehículo robótico controlado a distancia.

Funcionalidad y requerimientos

El sistema deberá cumplir con las siguientes funcionalidades:

- Investigar, analizar y comprender los parámetros clave del modelo MediaPipe Google, con un enfoque específico en la optimización de los FPS para asegurar un rendimiento fluido, el manejo de la detección del fondo para evitar falsas activaciones, y la correcta interpretación de los 21 puntos de referencia de la mano para una clasificación precisa de cada gesto.
- Implementar el modelo de detección de palma de MediaPipe.
- Reconocer al menos cinco gestos estáticos: por ejemplo, mano abierta, puño, gesto “OK”, dedo índice apuntando, y fondo (sin mano detectada).
- Mostrar la imagen de la cámara en una interfaz web, incluyendo la sobreimpresión de los puntos de referencia y la visualización de la cantidad de FPS alcanzados.
- Enviar los gestos reconocidos al NodeMCU, encargado de controlar un auto con dos servomotores 360° y una rueda loca de apoyo.
- Reflejar cada gesto en una acción específica del auto.

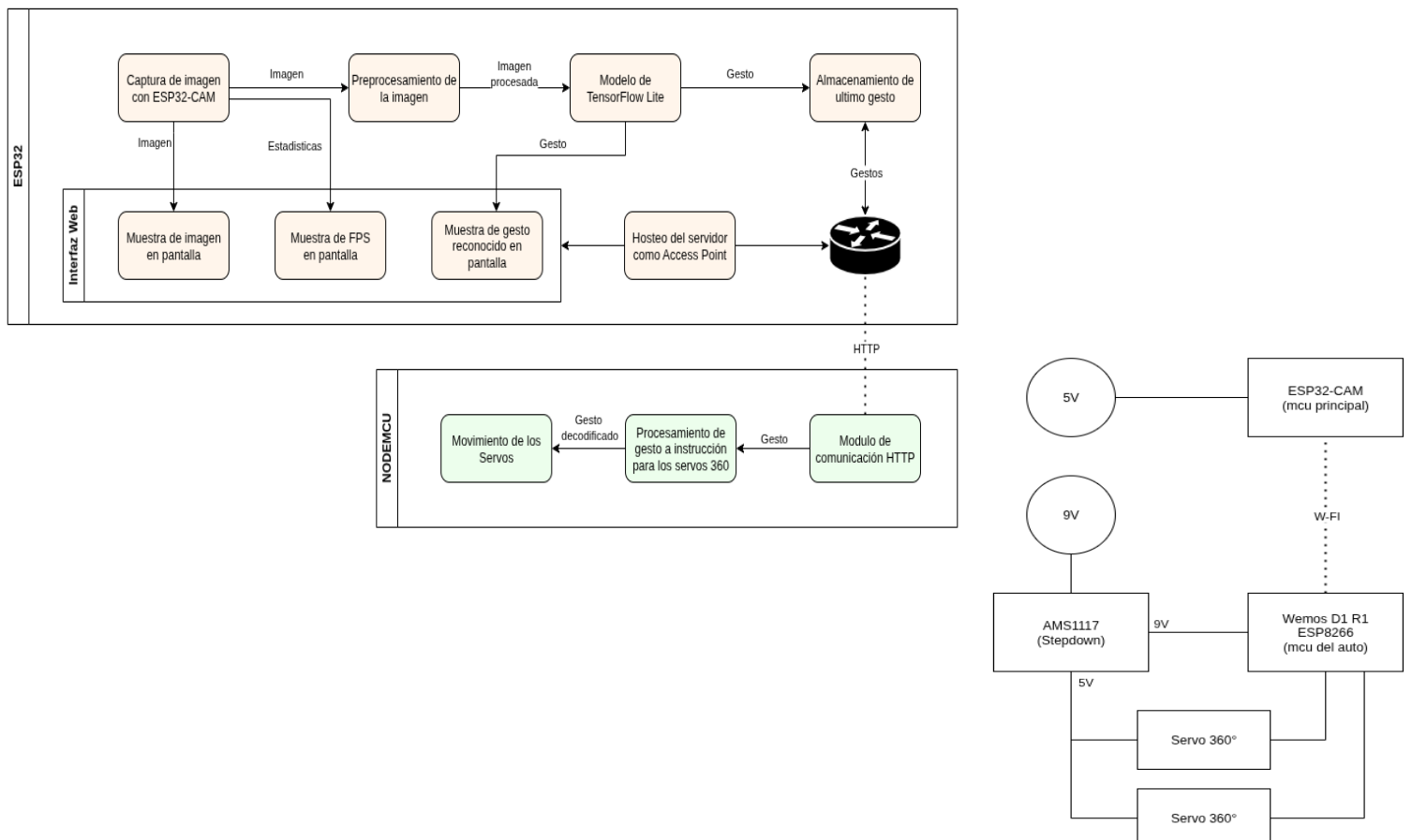
MATERIALES Y PRESUPUESTO

Tabla 1: Componentes de hardware necesarios y links directos para la compra.

Artículo	Cantidad	Precio	Link	Subtotal
ESP32 CAM	1	14500	[Link]	14500
Wemos D1 R1	1	11067	[Link]	11067
Servo 360°	2	8130	[Link]	16260
Rueda universal	1	4999	[Link]	4999
Fuente 9V 1A	1	7299	[Link]	7299
Fuente 5V 1,2A	1	8968	[Link]	8968
Stepdown (AMS1117)	2	5664	[Link]	11328
Total				74421

Todos los materiales necesarios fueron otorgados por la cátedra, incluso el auto ya armado. Queda a la espera la fuente de 5V para la ESP32-CAM.

ESQUEMA GRÁFICO DEL PROYECTO



GRADO DE AVANCE, PROBLEMAS Y SOLUCIONES

Módulo del auto

El módulo del auto se encuentra en una fase avanzada de desarrollo. Faltando solo pruebas de comunicación en modo cliente para su finalización.

Hardware

Para el auto se utilizó una Placa de Desarrollo Wemos D1 R1 (basado en ESP8266), con consumo medio. Además, dos servomotores de rotación continua, también con consumo medio.

Se realizó el diseño de la conexión necesaria para el módulo del auto utilizando el programa KiCAD y posterior soldado de una placa perforada para realizar las conexiones entre el Wemos D1, los servos y la alimentación.

Firmware

El firmware desarrollado para el control del auto se encuentra ubicado en la carpeta “*controladorAuto/*” del repositorio entregado por la cátedra.

Primero se desarrolló el controlador principal del vehículo en el Wemos D1 R1, creando las funciones para gestionar los movimientos básicos como avanzar, retroceder, girar y detenerse. El código principal de los motores se encuentra en el archivo **MotorControl.cpp**. Teniendo en cuenta que en los servos de rotación continua 90 grados es detener y 0 y 180 grados son velocidades máximas en direcciones opuestas.

```
1  #include <Servo.h>
2  #include "config.h"
3  #include "MotorControl.h"
4
5  // Crear objetos Servo para cada motor
6  Servo servoLeft;
7  Servo servoRight;
8
9  void auto_init() {
10     // Conectar los servos
11     servoLeft.attach(SERVO_LEFT_PIN);
12     servoRight.attach(SERVO_RIGHT_PIN);
13
14     // Asegurarse de que los motores estén detenidos al iniciar
15     auto_detener();
16 }
17
18 void auto_detener() {
19     servoLeft.write(90);
20     servoRight.write(90);
21 }
22
23 void auto_avanzar() {
24     // Para avanzar, ambos servos giran en direcciones opuestas
25     servoLeft.write(180);
26     servoRight.write(0);
27 }
28
29 void auto_reversa() {
30     // Para ir en reversa, se invierte la dirección de avance
31     servoLeft.write(0);
32     servoRight.write(180);
33 }
34
35 void auto_giro_horario() {
36     // Para girar en sentido horario, ambos motores giran al mismo lado
37     servoLeft.write(180);
38     servoRight.write(180);
39 }
40
41 void auto_giro_antihorario() {
42     // Para girar en sentido antihorario, se invierte la dirección del giro
43     servoLeft.write(0);
44     servoRight.write(0);
45 }
```

A continuación, se implementó la comunicación HTTP. Como la ESP CAM se estaba utilizando para otra sección del proyecto, se comenzó configurando el Wemos para operar temporalmente en modo Access Point. Esto permitió que el módulo creara su propia red Wi-Fi y levantara un servidor web capaz de enviar comandos, simulando los que se deben recibir por el otro módulo. El archivo **WifiHandler.cpp** se encarga de la inicialización y manejo de la comunicación Wi-Fi.

```
1  #include <ESP8266WebServer.h>
2  #include <ESP8266WiFi.h>
3  #include "config.h"
4  #include "WiFiHandler.h"
5  #include "webpage.h"
6
7  // --- Variables para el servidor en MODO AP ---
8  ESP8266WebServer server(80);
9  String receivedCommandAP = "";
10
11 // --- Funciones para manejar las peticiones del servidor ---
12 void handleRoot() {
13     if (server.hasArg("cmd")) {
14         receivedCommandAP = server.arg("cmd");
15     }
16     server.send(200, "text/html", HTML_CONTENT);
17 }
18
19 void handleNotFound() {
20     server.send(404, "text/plain", "404: Not Found");
21 }
22
23
24 // --- Implementación de las funciones principales ---
25
26 void wifi_init() {
27     // ---- INICIAR EN MODO ACCESS POINT (AP) ----
28     Serial.println("Iniciando en MODO ACCESS POINT...");
29
30     IPAddress local_IP(AP_SERVER_IP);
31     IPAddress gateway(AP_SERVER_IP);
32     IPAddress subnet(255, 255, 255, 0);
33
34     WiFi.softAPConfig(local_IP, gateway, subnet);
35     WiFi.softAP(AP_WIFI_SSID, AP_WIFI_PASSWORD);
36
37     Serial.print("AP iniciado. Conéctate a la red: ");
38     Serial.println(AP_WIFI_SSID);
39     Serial.print("IP del servidor: http://");
40     Serial.println(WiFi.softAPIP());
41
42     server.on("/", handleRoot);
43     server.onNotFound(handleNotFound);
44     server.begin();
45     Serial.println("Servidor HTTP iniciado.");
46 }
47
48 String wifi_get_command() {
49     receivedCommandAP = "";
50     server.handleClient();
51     return receivedCommandAP;
52 }
```


Todo el código fue modularizado para una mejor organización. Se separaron las funciones en distintos archivos: *MotorControl* para el manejo de los servos, *WiFiHandler* para la gestión de la red y el servidor, *config.h* para los parámetros globales, y el *ControladorAuto.ino* como código principal.

```
1  #include "config.h"
2  #include "MotorControl.h"
3  #include "WiFiHandler.h"
4
5
6  void setup() {
7      Serial.begin(115200);
8      Serial.println("\nIniciando controlador del auto...");
9
10     // Inicializa los motores
11     auto_init();
12
13     // Inicializa el WiFi
14     wifi_init();
15
16     Serial.println("Sistema listo.");
17 }
18
19
20 void loop() {
21
22     // Se obtiene el siguiente comando desde el servidor web
23     String command = wifi_get_command();
24
25     // Si se recibió un comando, procesarlo
26     if (command.length() > 0) {
27         Serial.print("Comando recibido: ");
28         Serial.println(command);
29
30         if (command == CMD_AVANZAR) {
31             auto_avanzar();
32         } else if (command == CMD_REVERSA) {
33             auto_reversa();
34         } else if (command == CMD_GIRO_DERECHA) {
35             auto_giro_horario();
36         } else if (command == CMD_GIRO_IZQUIERDA) {
37             auto_giro_antihorario();
38         } else if (command == CMD_DETENER) {
39             auto_detener();
40         }
41     }
42 }
43
```

Informe de Avance A4 - Reconocimiento de gesto estático

El archivo **config.h** se usó para poder cambiar en caso de ser necesario las direcciones o contraseñas de red, los comandos para el control del auto o los pines a los que se conectan los servos.

```
1  #ifndef CONFIG_H
2  #define CONFIG_H
3
4  // ===== CONFIGURACIÓN DE PINES =====
5  // ==
6  // =====
7  #define SERVO_LEFT_PIN D7 // Pin para el servo izquierdo
8  #define SERVO_RIGHT_PIN D6 // Pin para el servo derecho
9
10
11 // ===== CONFIGURACIÓN MODO ACCESS POINT (AP) =====
12 // ==
13 // =====
14 #define AP_WIFI_SSID "AutoESP8266" // Nombre de la red WiFi que creará el Wemos
15 #define AP_WIFI_PASSWORD "12345678" // Contraseña de la red
16 #define AP_SERVER_IP 10,10,10,10 // IP estática del servidor web
17
18
19 // ===== DEFINICIÓN DE COMANDOS HTTP =====
20 // ==
21 // =====
22 #define CMD_AVANZAR "AVANZAR"
23 #define CMD_DETENER "DETENER"
24 #define CMD_REVERSA "REVERSA"
25 #define CMD_GIRO_DERECHA "GIRO_DERECHA"
26 #define CMD_GIRO_IZQUIERDA "GIRO_IZQUIERDA"
27
28 #endif // CONFIG_H
```

Se generó una aplicación web simple, contenida en el archivo **webpage.h**, que presenta una interfaz con botones. Al interactuar con estos botones, se envían los comandos predefinidos al servidor del Wemos mediante JavaScript, permitiendo el control del vehículo. Para los botones de movimiento se incluyeron los eventos “onmouse” y “ontouch” para que los botones reaccionen tanto para PC como para celulares y que al presionar el botón se inicie el movimiento y al soltarlo se frene con los eventos correspondientes.

```
1  #ifndef WEBPAGE_H
2  #define WEBPAGE_H
3
4  const char HTML_CONTENT[] PROGMEM = R"====="
5  <!DOCTYPE html>
6  <html lang="es">
7  <head>
8  <meta charset="UTF-8">
9  <meta name="viewport" content="width=device-width,initial-scale=1,user-scalable=no">
10 <title>Control Auto Wemos</title>
11 <style>
12 html,body{height:100%;margin:0;display:flex;flex-direction:column;justify-content:center;align-items:center;background-color:#f0f0f0;font-family:sans-serif;touch-action:manipulation}
13 h1{margin-bottom:2rem;font-size:2rem;text-align:center}
14 .control-panel{display:grid;grid-template-columns:repeat(3,1fr);grid-template-rows:repeat(3,1fr);gap:15px;width:90vw;max-width:380px;height:90vw;max-height:380px}
15 .control-button{display:flex;justify-content:center;align-items:center;border-radius:12px;font-size:2.5rem;color:#cf0f1;font-weight:bold;transition:background-color .2s ease,transform .1s ease}
16 .control-button:active{transform:scale(.95)}
17 #forward{grid-area:1 / 2 / 2 / 3}
18 #left{grid-area:2 / 1 / 3 / 2}
19 #stop{grid-area:2 / 2 / 3 / 3}
20 #right{grid-area:2 / 3 / 3 / 4}
21 #reverse{grid-area:3 / 1 / 4 / 3}
22 .direction-btn{background-color:#288b99}
23 .direction-btn:active{background-color:#3498db}
24 #stop{background-color:#c3922b;font-size:1.5rem;font-weight:700}
25 #stop:active{background-color:#e74c3c}
26 </style>
27 </head>
28 <body>
29 <h1>Control Auto Wemos</h1>
30 <div class="control-panel">
31 <div id="forward" class="control-button direction-btn" onmousedown="sendCommand('AVANZAR')" onmouseup="sendCommand('DETENER')" ontouchstart="event.preventDefault();sendCommand('AVANZAR')" ontouchend="sendCommand('DETENER')"></div>
32 <div id="left" class="control-button direction-btn" onmousedown="sendCommand('GIRO_IZQUIERDA')" onmouseup="sendCommand('DETENER')" ontouchstart="event.preventDefault();sendCommand('GIRO_IZQUIERDA')" ontouchend="sendCommand('DETENER')"></div>
33 <div id="stop" class="control-button" onclick="sendCommand('DETENER')">STOP</div>
34 <div id="right" class="control-button direction-btn" onmousedown="sendCommand('GIRO_DERECHA')" onmouseup="sendCommand('DETENER')" ontouchstart="event.preventDefault();sendCommand('GIRO_DERECHA')" ontouchend="sendCommand('DETENER')"></div>
35 <div id="reverse" class="control-button direction-btn" onmousedown="sendCommand('REVERSA')" onmouseup="sendCommand('DETENER')" ontouchstart="event.preventDefault();sendCommand('REVERSA')" ontouchend="sendCommand('DETENER')"></div>
36 </div>
37 <script>
38 function sendCommand(c){fetch("/?cmd="+c)}
39 </script>
40 </body>
41 </html>
42 )====="
43
44 #endif // WEBPAGE_H
```

Finalmente, se realizaron pruebas de funcionamiento del sistema en esta configuración. Se validó con éxito que el vehículo responde correctamente a los comandos enviados desde un cliente web (se probó desde el teléfono y computadora) conectado a la red Wi-Fi del Wemos.

Módulo ESP32-CAM

Hardware

Se diseñó la reestructuración de la placa perforada entregada por la cátedra. Se configuró la misma, desoldando las conexiones necesarias y soldando las nuevas.

Interfaz web

Para la visualización y el control del sistema de reconocimiento de gestos, se desarrolló una interfaz web preliminar. El objetivo de esta interfaz es ofrecer al usuario una experiencia clara y funcional, permitiéndole ver en tiempo real la imagen capturada por la cámara, el gesto que está siendo reconocido y las instrucciones de uso.

La interfaz se construyó utilizando tecnologías web estándar para garantizar la compatibilidad y simplicidad:

- **HTML:** Se utilizó para estructurar el contenido de la página, definiendo las secciones principales: un encabezado, un área para el video y el estado del gesto, y una sección con las instrucciones.
- **CSS:** Se empleó para el diseño visual. Se creó una estética limpia y moderna, con un diseño adaptable.
- **JavaScript:** Se encarga de gestionar el acceso a la cámara, actualizar el estado del gesto y, en la versión final, de la comunicación con el microcontrolador.

Dado que esta es una fase de desarrollo inicial en la que el software de reconocimiento aún no está realizado y la ESP32 no estaba disponible para utilizar ya que se encontraba realizando pruebas en otra sección del proyecto, se implementaron las siguientes funcionalidades a modo de prototipo:

- **Visualización de Video:** Para simular el *stream* de video que provendrá de la ESP32-CAM, se utilizó la API `navigator.mediaDevices.getUserMedia` de JavaScript. Esto permite acceder a la cámara web del computador donde se abre la página. Esta implementación es un sustituto temporal que nos permite desarrollar y probar el diseño de la interfaz sin depender del hardware final.

- **Simulación de Reconocimiento de Gestos:** Como el código de reconocimiento de gestos aún no está integrado, se simuló la recepción de datos. Mediante la función `setInterval` de JavaScript, la interfaz alterna entre los diferentes gestos posibles ("Dedo Índice Arriba", "Mano Abierta", etc.) cada 3 segundos, mostrando el texto correspondiente en el apartado "Gesto Reconocido". Esto permite verificar el correcto funcionamiento del componente visual que informará al usuario sobre el estado del sistema.
- **Panel de Instrucciones:** Se diseñó un panel lateral estático que muestra de forma clara y concisa los gestos disponibles y la acción que desencadena cada uno en el vehículo (avanzar, detenerse, etc.). Esto asegura que el usuario siempre tenga a mano la información necesaria para operar el sistema.

El archivo *index.html* contiene toda la lógica y se puede visualizar en [la carpeta /esp32cam/interfazWeb del repositorio del proyecto en GitHub.](#)

Para que la interfaz sea completamente funcional y se integre con el sistema final, es necesario abordar los siguientes puntos:

1. **Integración del Stream de la ESP32-CAM:** Se deberá reemplazar el acceso a la cámara local por la fuente de video proveniente del servidor de la ESP32-CAM.
2. **Conexión vía WebSocket:** Se debe implementar la comunicación entre la interfaz web (cliente) y la ESP32 (servidor) utilizando **WebSockets**. El código base ya está preparado pero comentado. Esta conexión permitirá recibir los datos del gesto reconocido en tiempo real, en lugar de usar la simulación actual.
3. **Visualización de Puntos de Referencia:** La tarea más compleja será recibir las coordenadas de los 21 puntos de referencia de la mano detectados por el modelo de IA en la ESP32.

Reconocimiento de gestos

De acuerdo con lo planificado en el Plan de Proyecto presentado, el desarrollo del modelo reconocedor de gestos de la mano se encuentra en una fase intermedia. Esto constituye la parte más compleja y fundamental de nuestro proyecto, por lo que se dedicó bastante tiempo en esta etapa inicial en investigación y aprendizaje de MediaPipe, estudiando la documentación sobre la generación de los 21 puntos de referencia en la mano y los mecanismos de detección de gestos. Todos los códigos de investigación y pruebas de uso de MediaPipe se encuentran en la carpeta "*mediapipe*" del repositorio entregado por la cátedra.

Se instaló y configuró la librería de MediaPipe en Python, realizando pruebas de reconocimiento de gestos utilizando la cámara de la computadora. Se logró ejecutar un código de python que permite superponer los 21 puntos de la mano sobre la imagen y reconocer el gesto hecho por la misma. El mismo se encuentra en la carpeta “*mediapipe/mediapipe_gestureAndPoints/*” del repositorio. Para ejecutar el código se puede utilizar el comando “*\$ docker compose up*” posicionado en la carpeta nombrada, o ejecutar el archivo “*rebuild.sh*”.

Se estableció un entorno de desarrollo en Google Colab que integra los puntos de referencia de MediaPipe para reconocimiento de manos con procesamiento de imágenes, donde se desarrolló la detección de dos gestos fundamentales: mano abierta (palma) y mano cerrada (puño), el sistema identifica los puntos clave de la mano y calcula las distancias entre ellos para determinar si los dedos están extendidos o contraídos. Una vez comprendida la lógica de puntos y distancias, se extendió el sistema para detectar dos gestos adicionales: "paz" y "OK", obteniendo muy buenos resultados en precisión y posición en general. El código de este reconocimiento se encuentra en la carpeta “*mediapipe/puntosMP/*” del repositorio.

El principal problema detectado es la limitación de procesamiento de la ESP32, al ejecutar programas exigentes en memoria y CPU, el dispositivo falla; para solucionar esta limitación, se integrará MediaPipe en un modelo de Tensor Flow, lo que permitirá aligerar significativamente la carga de procesamiento.

Se ha comenzado la creación del modelo de Tensor Flow específico para reconocer gestos y puntos de referencia de la mano, es por ello por lo que se inició la organización de un Dataset en Google Drive con imágenes de referencia clasificadas y agrupadas según los gestos específicos. Las imágenes están siendo capturadas utilizando la ESP-32 para replicar las condiciones reales de ejecución de la aplicación final.

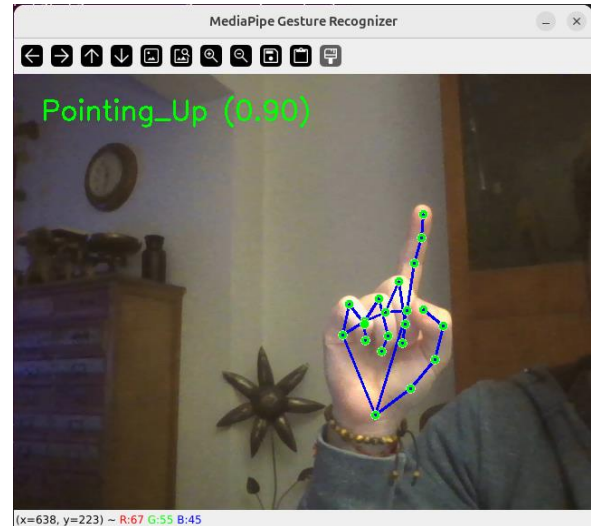
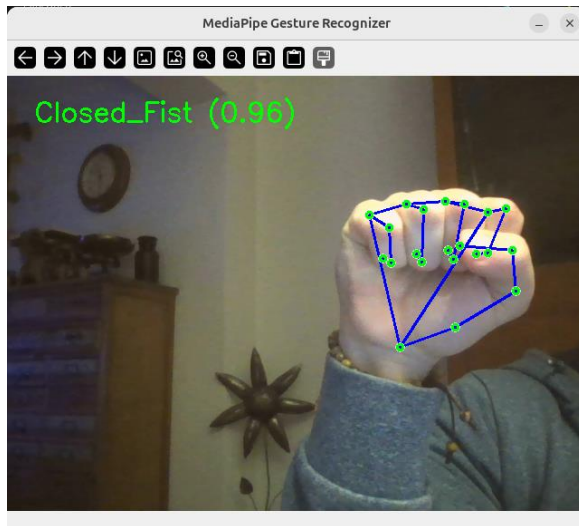
Si bien hasta el momento no hemos encontrado problemas significativos, hemos identificado desafíos previsibles como que el modelo podría tener dificultades para detectar consistentemente los puntos de referencia de la mano en diferentes condiciones de iluminación, ángulos y fondos por lo que será crucial tener un dataset completo con variaciones en estos parámetros.

Para resolver los desafíos de detección, se dispone de la extensa documentación oficial de MediaPipe, que incluye la Guía de personalización del modelo de reconocimiento de gestos con la mano que resulta de gran ayuda y de guía para el desarrollo del proyecto.

DOCUMENTACIÓN RELACIONADA

Investigación y trabajo inicial

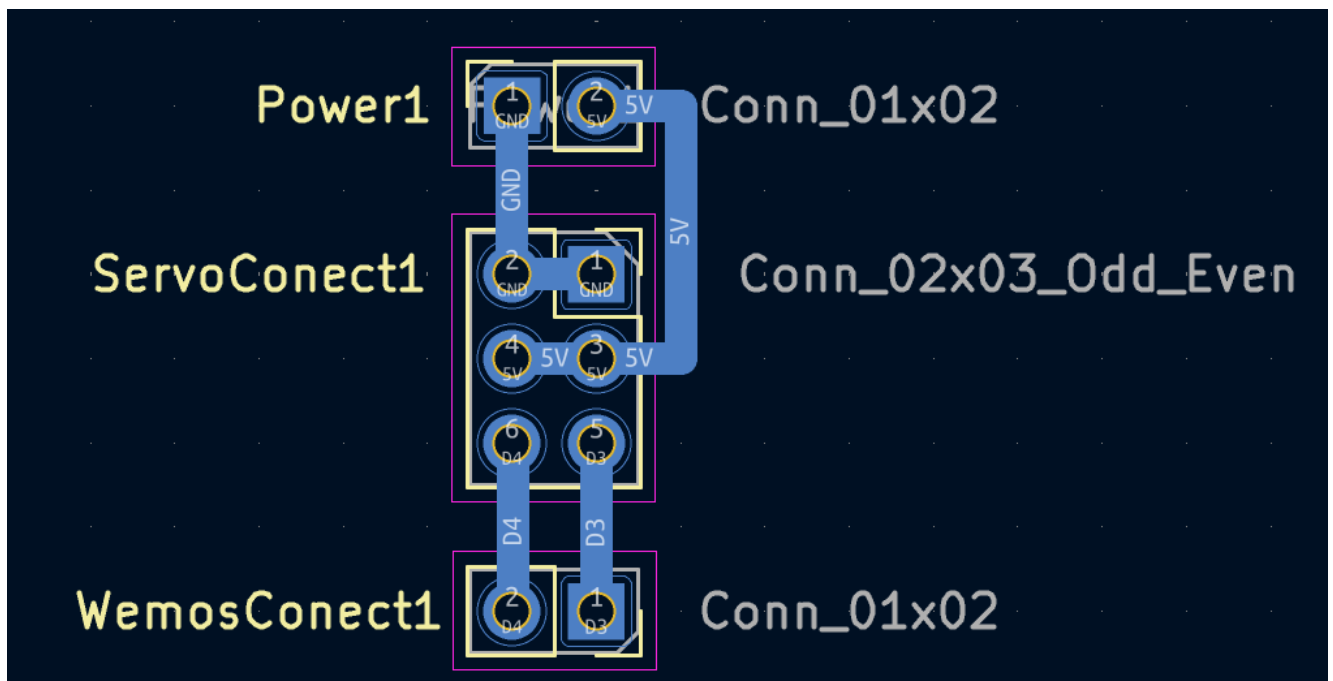
A continuación, se muestran dos imágenes del sistema de prueba de mediapipe funcionando, ejecutado en una PC.



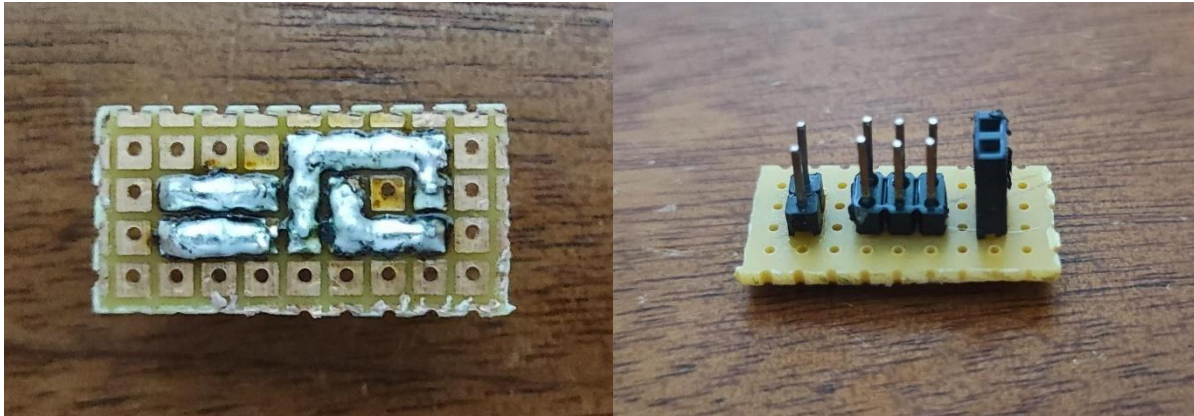
Módulo del auto

Hardware

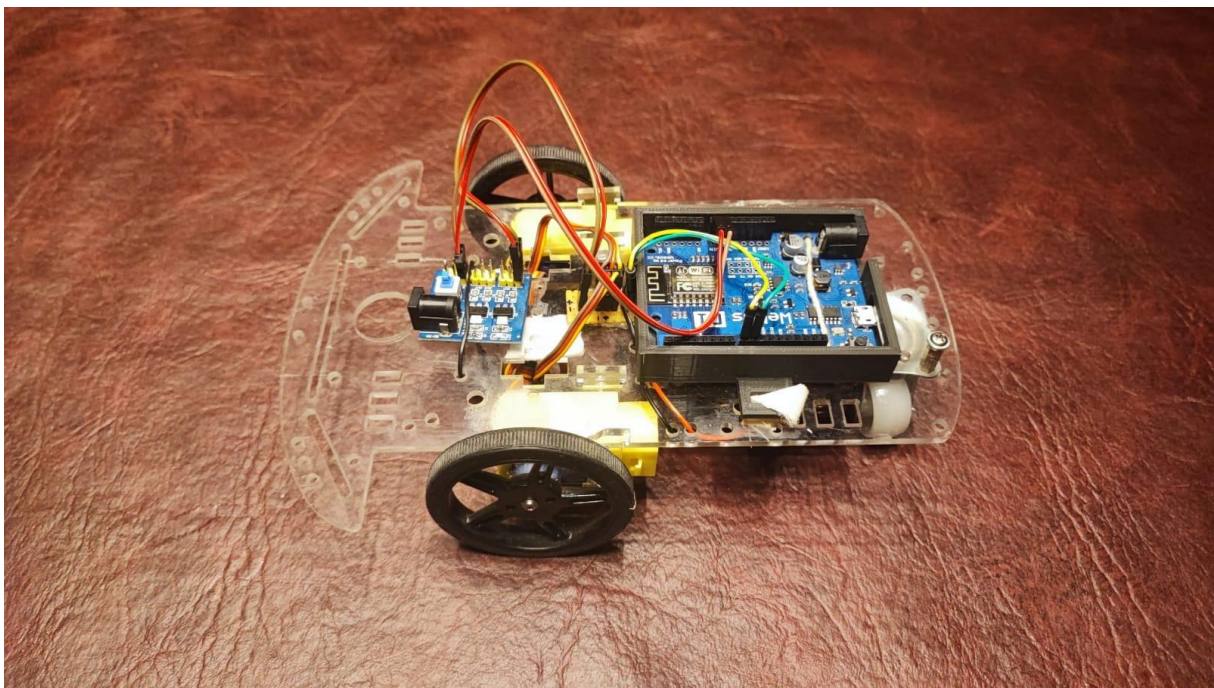
Diseño original de la conexión necesaria para la placa perforada en el programa KiCAD:



Imágenes de las placas soldadas para la conexión de los servos:

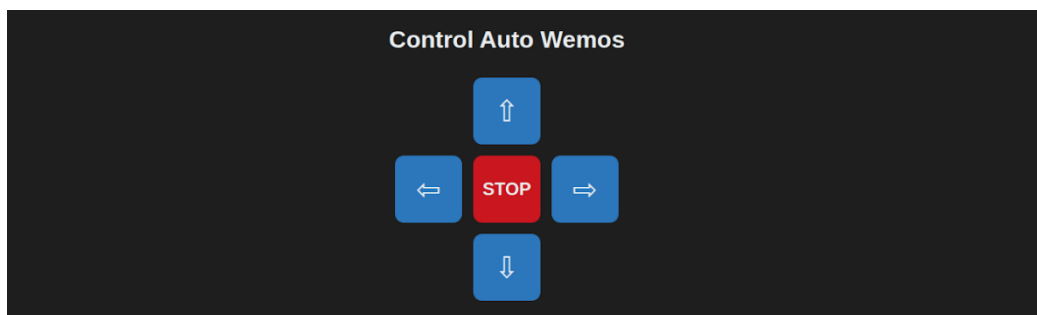


Diseño completo del auto:



Firmware

Interfaz web de control del auto, hecha únicamente para pruebas.

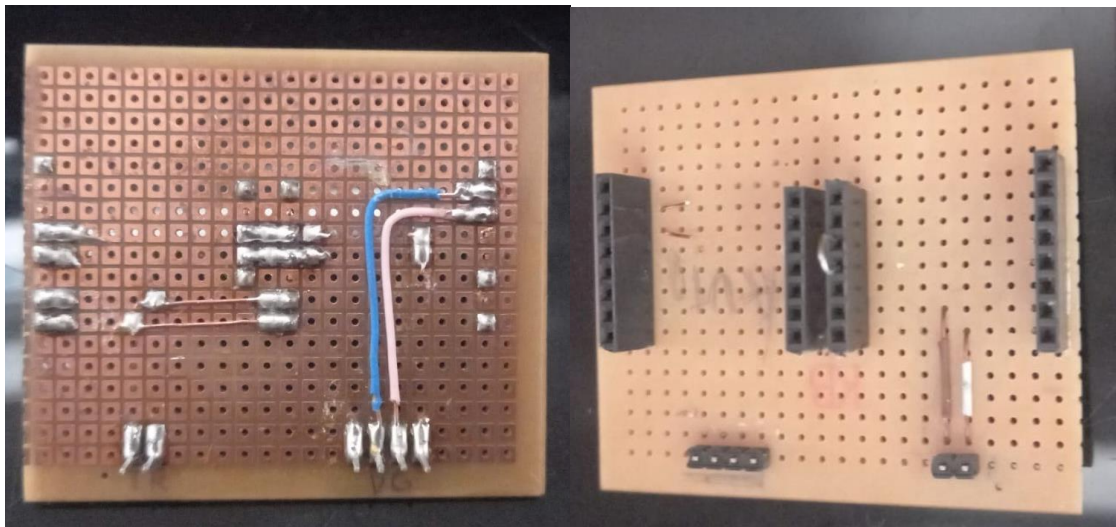


[Video de pruebas de funcionamiento](#)

Módulo ESP32-CAM

Hardware

Se rediseñó una de las placas perforadas entregadas por la cátedra para la conexión de la ESP32 CAM. Se cambió el cableado original por uno que conecte los pines necesarios para el proyecto. Además de dejar dos pines hembras donde conectar un jumper para poder hacer el flash de la ESP.



Interfaz web



Repositorio

Se puede encontrar todo el código mencionado a lo largo de este informe en el [repositorio de GitHub del proyecto](#), específicamente en la rama “*entrega_octubre*”. También es posible acceder a la bitácora del grupo mediante [este enlace](#).